6.5830/1 Lecture 4: Database Internals

2/12/2026

Tianyu Li

litianyu@mit.edu

Many thanks to Andy Pavlo and CMU DB for sharing their materials

Administrivia

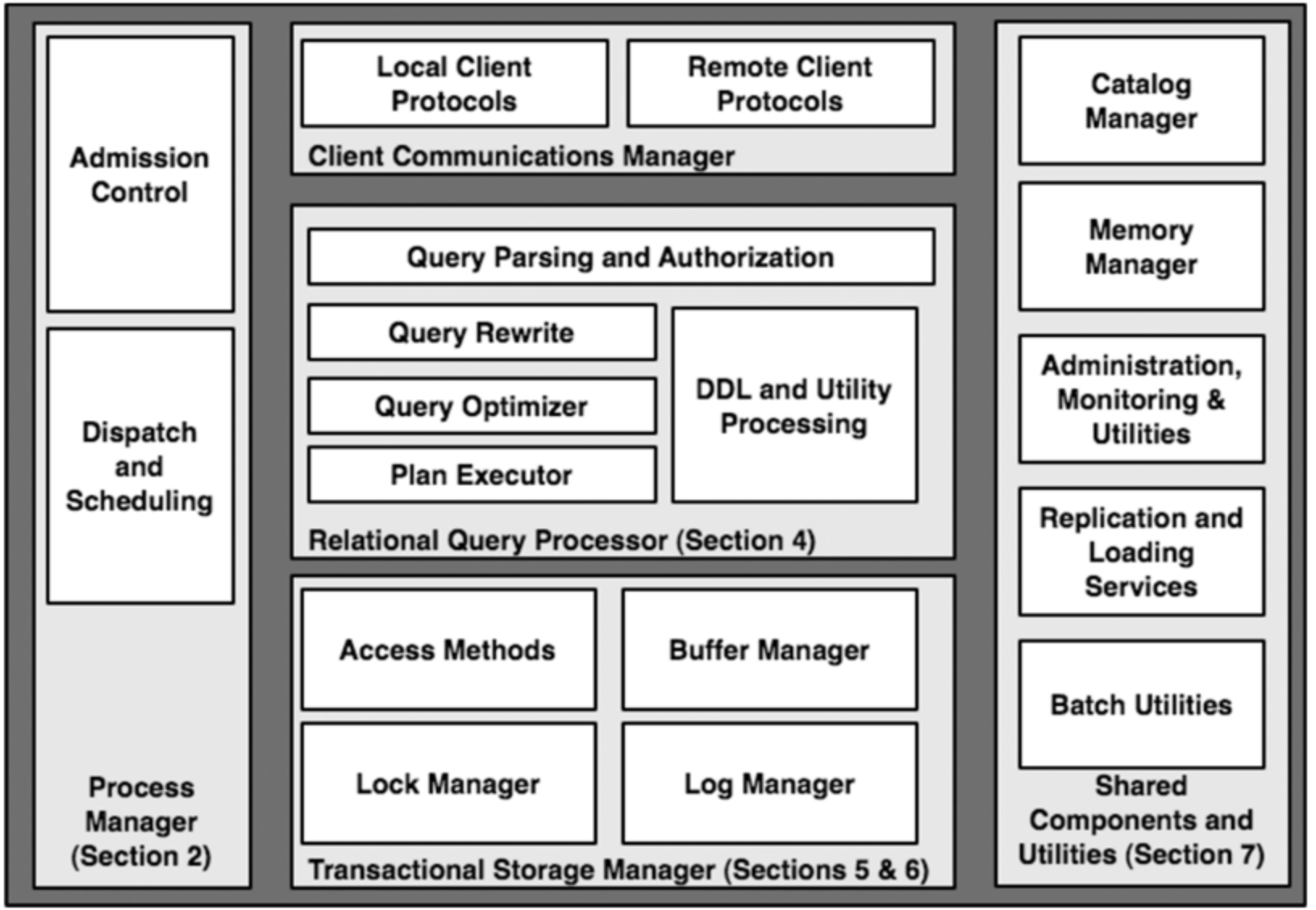
- Pset 1 due in one week, Lab 1 is out, due in two weeks
  - We will cover most relevant materials today
  - TAs will hold (and record) bootcamp
  - Start early and come to office hours!
- Start thinking about projects!
  - See list of suggested projects
- No class on Feb 17! (Monday schedule)
- Chroma (vector DB) talk on Feb 18
  - 32-G882 at 1pm, details on CSAIL event calendar

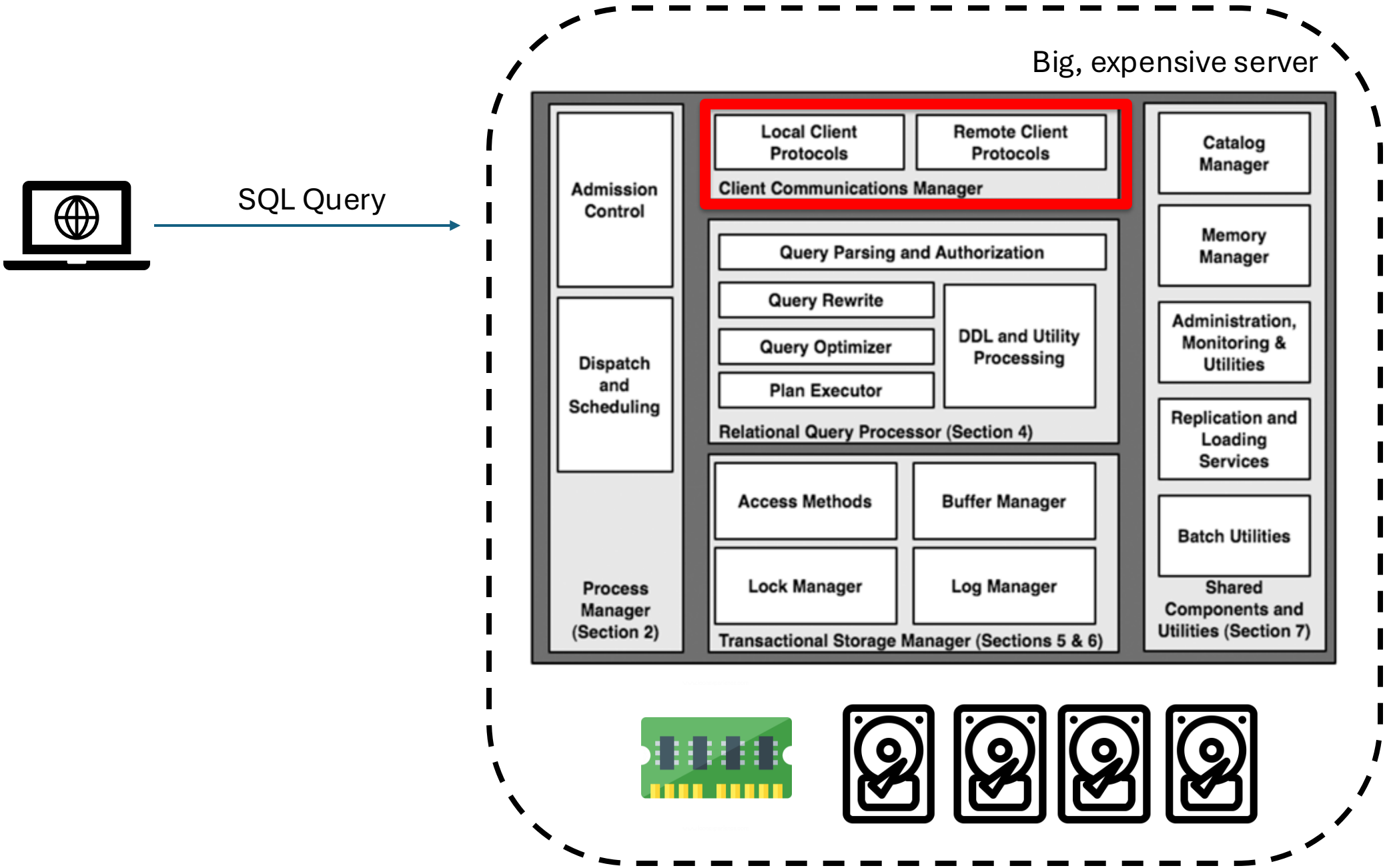
Recap

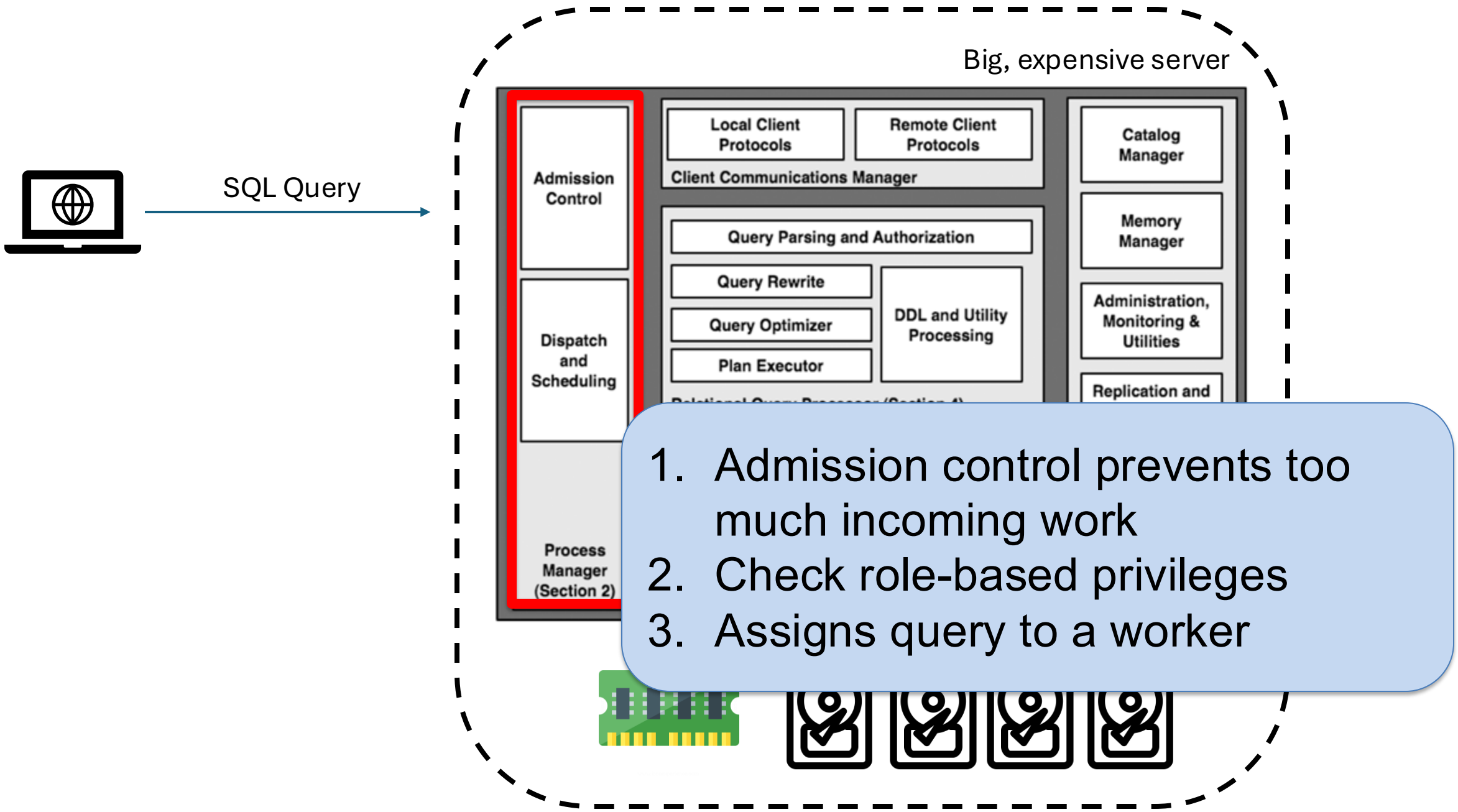
- Last lectures: relational model, schema design, SQL
  - Mental model + interface for database users
  - Many workloads mapped onto this conceptual model

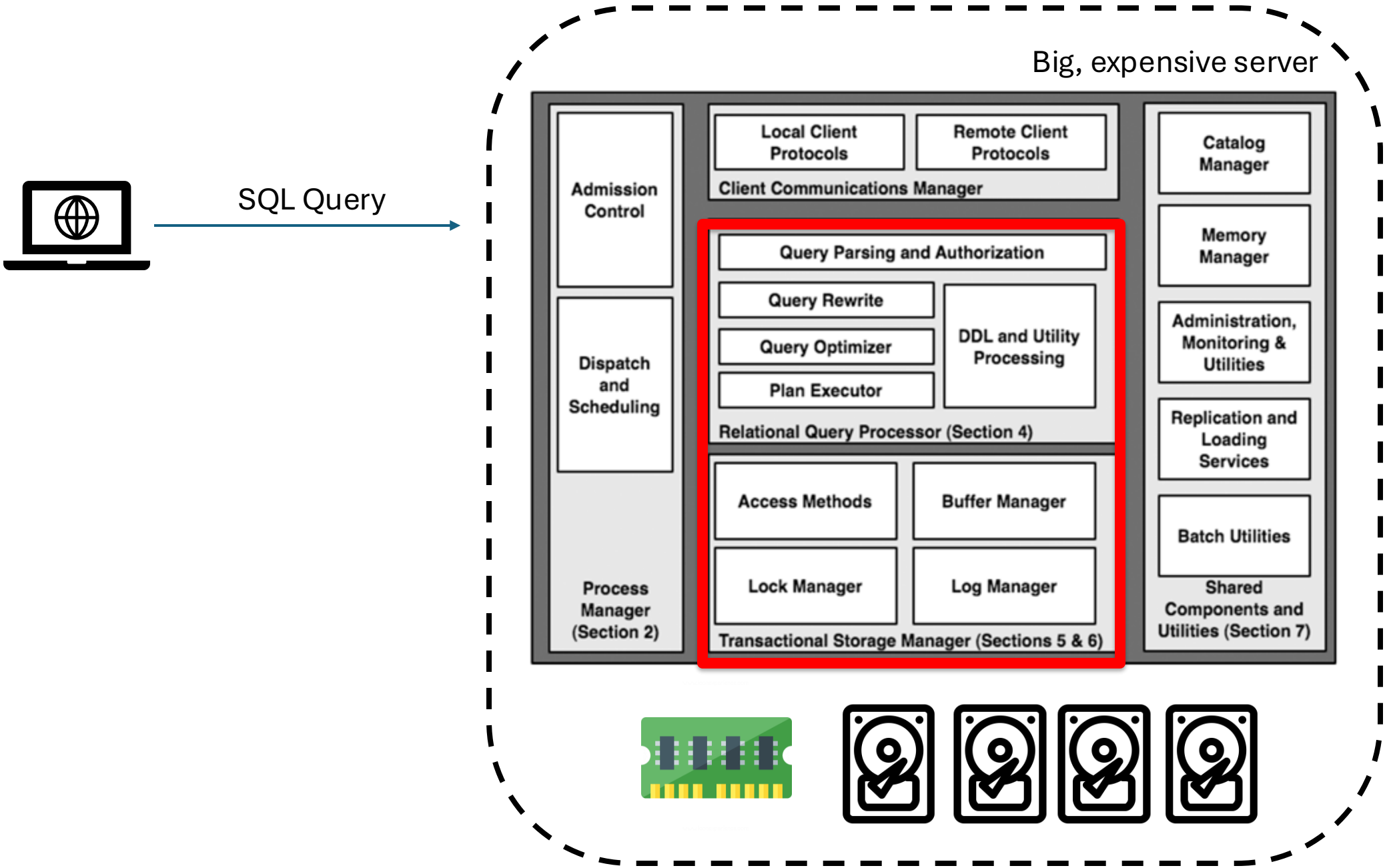
- Next: but how do we implement all this?
  - Performance
  - Engineering complexity
  - Safety under concurrency
  - Durability & fault-tolerance

Database System Architecture

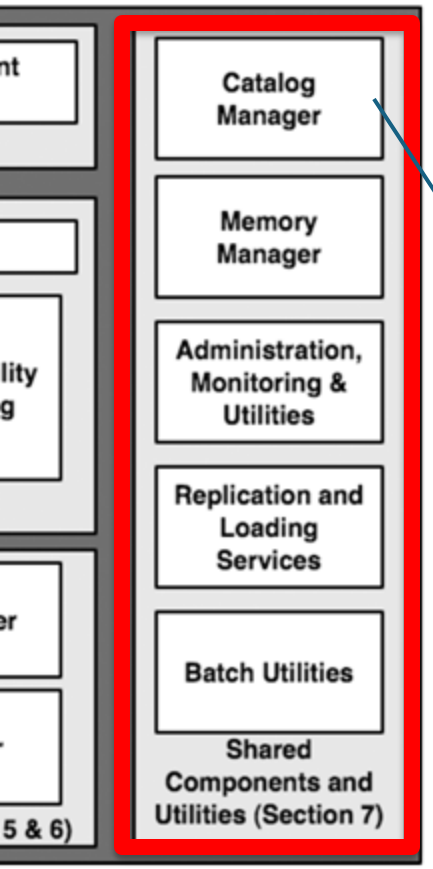


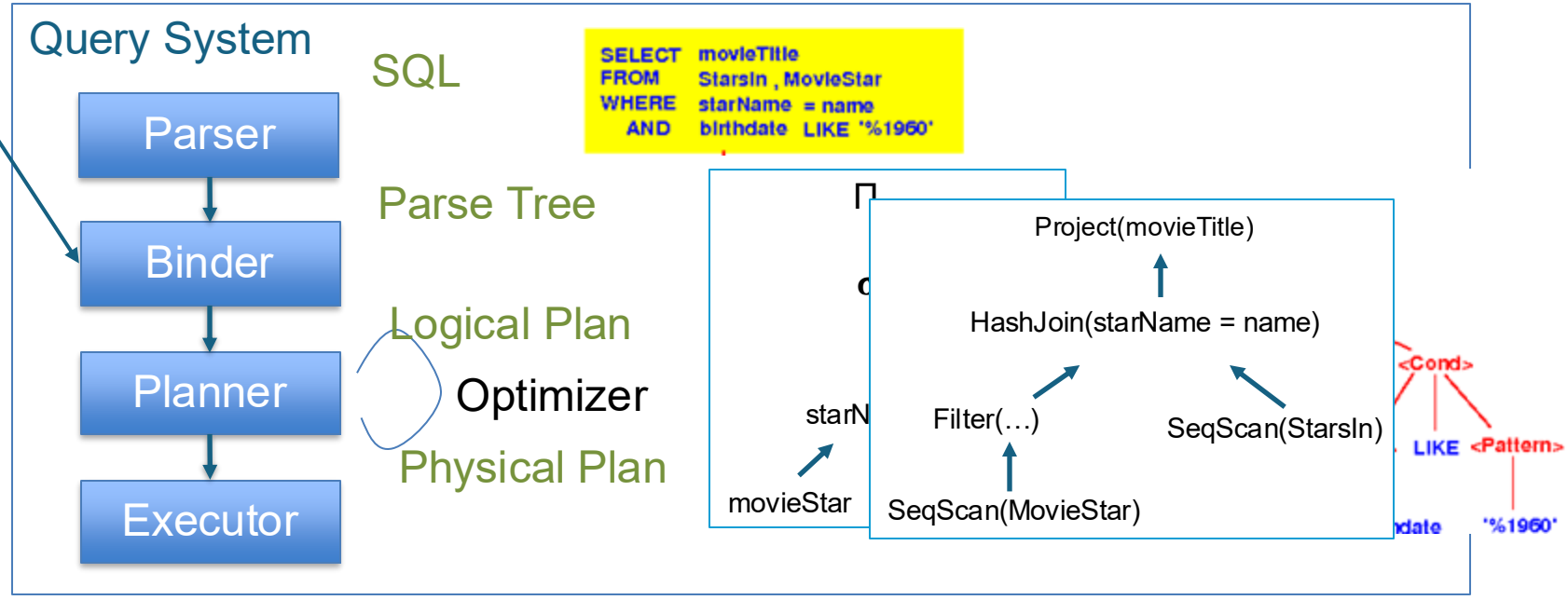

Big, expensive server
SQL Query
Admission Control
Dispatch and Scheduling
Process Manager (Section 2)
Local Client Protocols
Remote Client Protocols
Client Communications Manager
Query Parsing and Authorization
Query Rewrite
Query Optimizer
Plan Executor
DDL and Utility Processing
Relational Query Processor (Section 4)
Access Methods
Buffer Manager
Lock Manager
Log Manager
Transactional Storage Manager (Sections 5 & 6)
Catalog Manager
Memory Manager
Administration, Monitoring & Utilities
Replication and Loading Services
Batch Utilities
Shared Components and Utilities (Section 7)


Big, expensive server
SQL Query
Admission Control
Dispatch and Scheduling
Process Manager (Section 2)
Local Client Protocols
Remote Client Protocols
Client Communications Manager
Query Parsing and Authorization
Query Rewrite
Query Optimizer
Plan Executor
DDL and Utility Processing
Catalog Manager
Memory Manager
Administration, Monitoring & Utilities
Replication and
1. Admission control prevents too much incoming work
2. Check role-based privileges
3. Assigns query to a worker

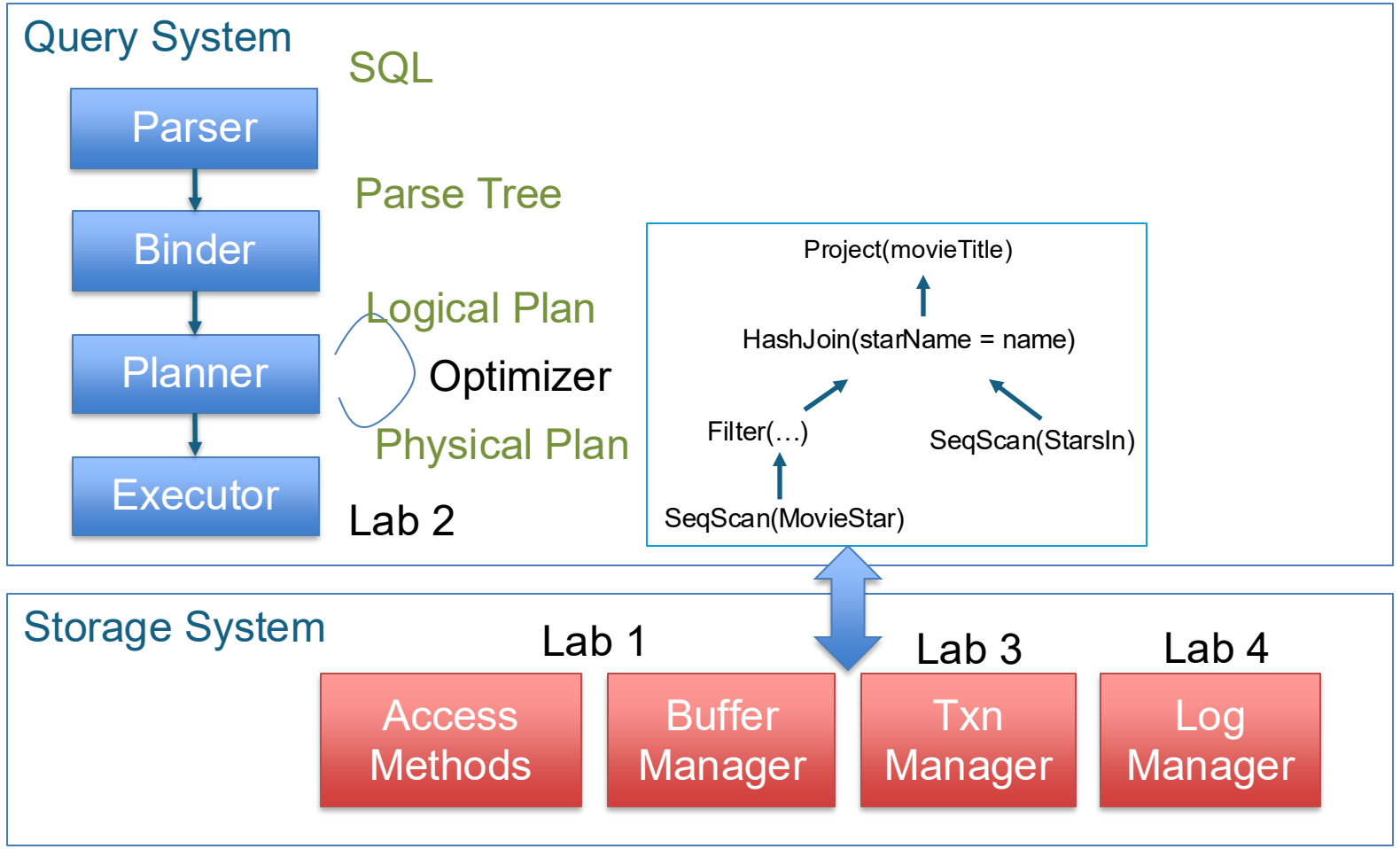

Big, expensive server
SQL Query
Admission Control
Dispatch and Scheduling
Process Manager (Section 2)
Local Client Protocols
Remote Client Protocols
Client Communications Manager
Query Parsing and Authorization
Query Rewrite
Query Optimizer
Plan Executor
DDL and Utility Processing
Relational Query Processor (Section 4)
Access Methods
Buffer Manager
Lock Manager
Log Manager
Transactional Storage Manager (Sections 5 & 6)
Catalog Manager
Memory Manager
Administration, Monitoring & Utilities
Replication and Loading Services
Batch Utilities
Shared Components and Utilities (Section 7)

Query Execution: Overview





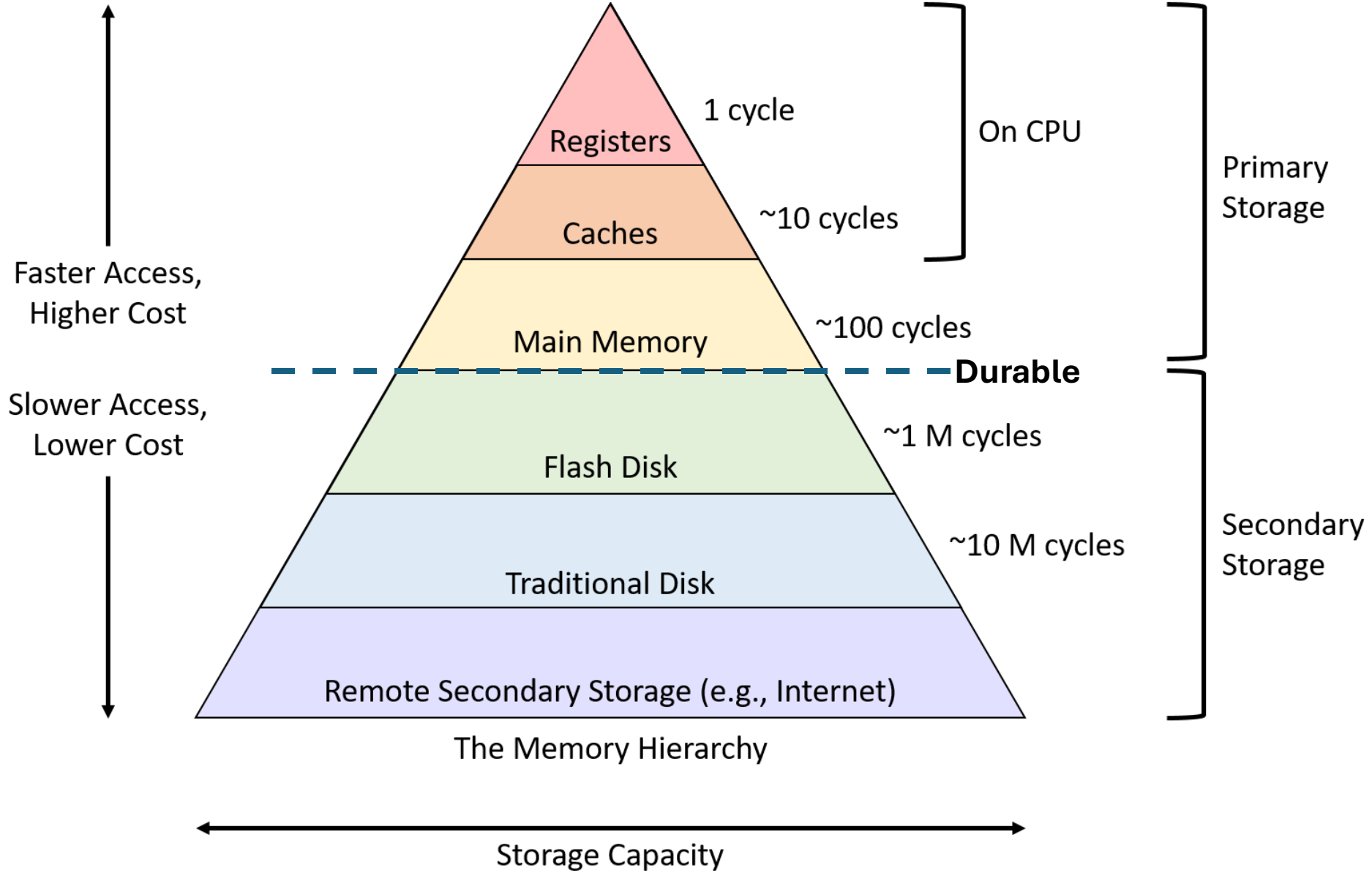
Query Execution: Overview



Today: Storage

- Traditional DBMS assumes that data is stored on non-volatile disks
  - HDD in the past, likely SSD or even remote storage today

- Key Tension:
  - Storage is slow but durable and plentiful
  - Memory is fast but scarce*
  - DBMS must manage the movement of data between them


Faster Access,
Higher Cost
Slower Access,
Lower Cost
Registers
1 cycle
Caches
~10 cycles
Main Memory
~100 cycles
Durable
Flash Disk
~1 M cycles
Traditional Disk
~10 M cycles
Remote Secondary Storage (e.g., Internet)
On CPU
Primary
Storage
Secondary
Storage
The Memory Hierarchy
Storage Capacity

Important Numbers

L1 latency	1 ns	1s
L2 latency	4 ns	4s
Main memory latency	100ns	~1.5 min
SSD Latency	16,000ns	~4.5 hours
HDD (spinning disk) latency	2,000,000 ns	~3.5 weeks
Network Storage	~50,000,000 ns	~1.5 years

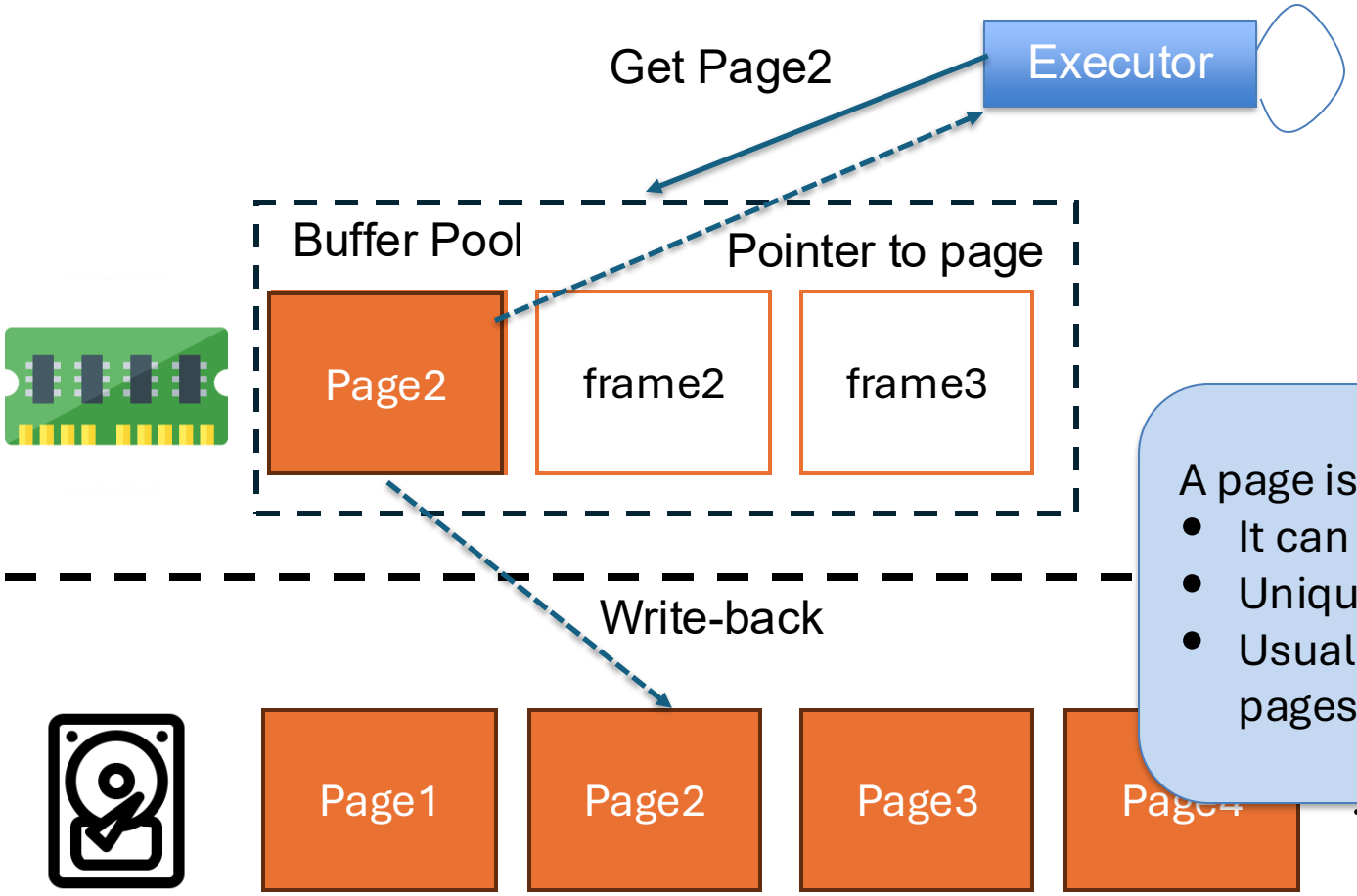
System Design Goals

- The DBMS must be able to manage databases that exceed the amount of memory available

- Must optimize slow disk access to avoid large stalls and performance problems
  - Minimize random access, prefer sequential access
  - Maximize reuse

Disk-Oriented Storage

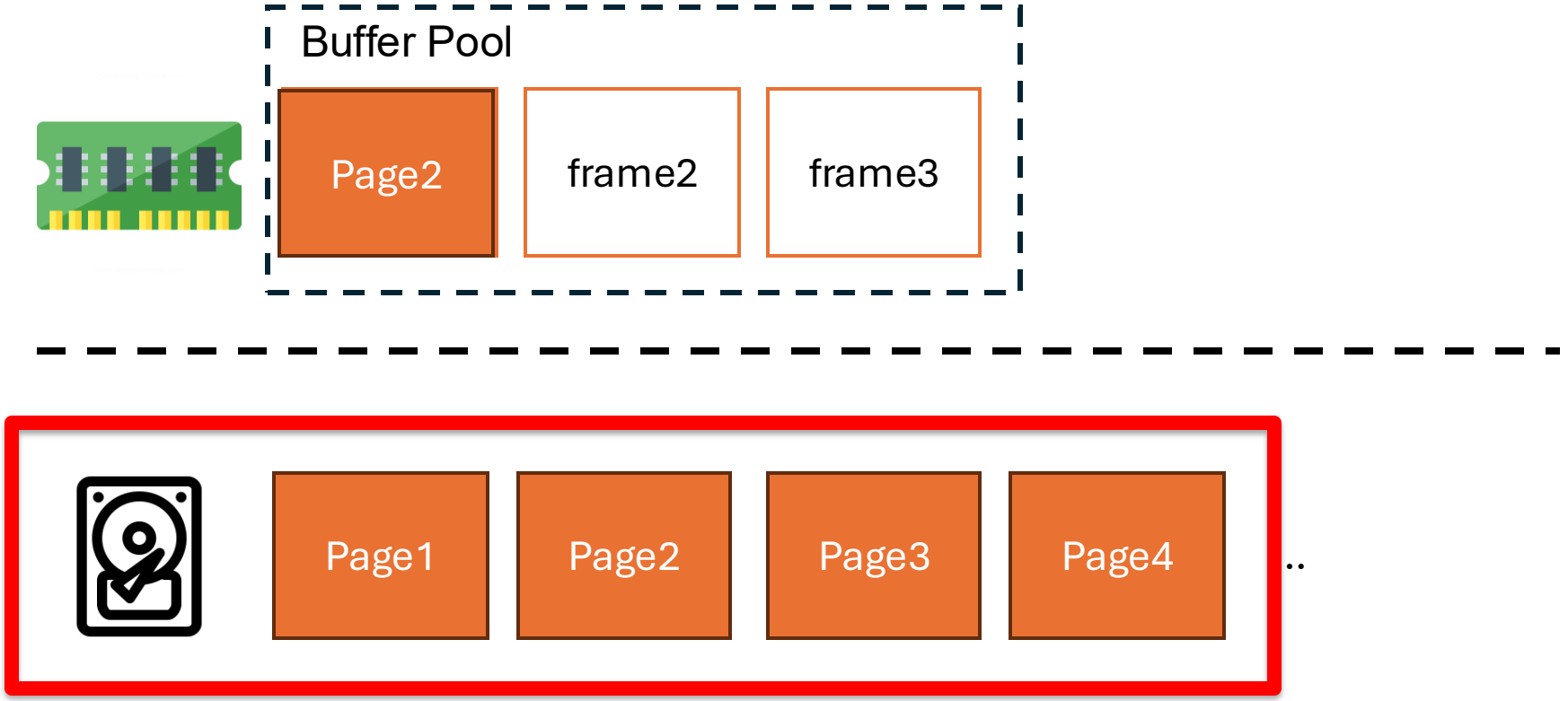
~ file system



A page is a fixed-size block of data

- It can contain tuples, metadata, etc.
- Uniquely identified by a PageID
- Usually 32 – 512 KB. GoDB uses 4 KB pages for simplicity

Disk-Oriented Storage

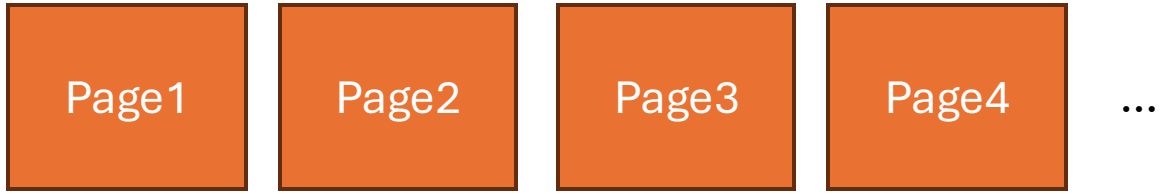


How does the DBMS represent data in files on disk?

Page Storage Architecture

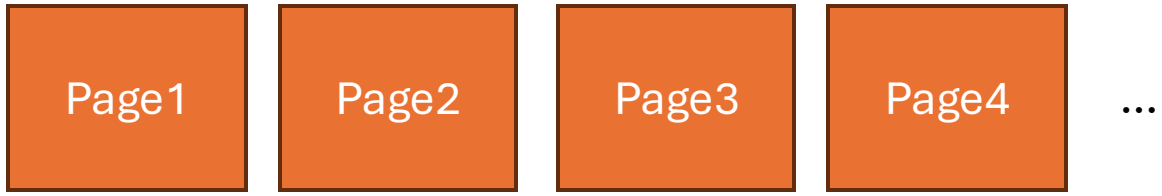
- How does a DBMS manage pages in files on disk?
  - As an unordered collection (i.e. Heap File)
  - Sorted (tree or otherwise)
  - Etc.

Today



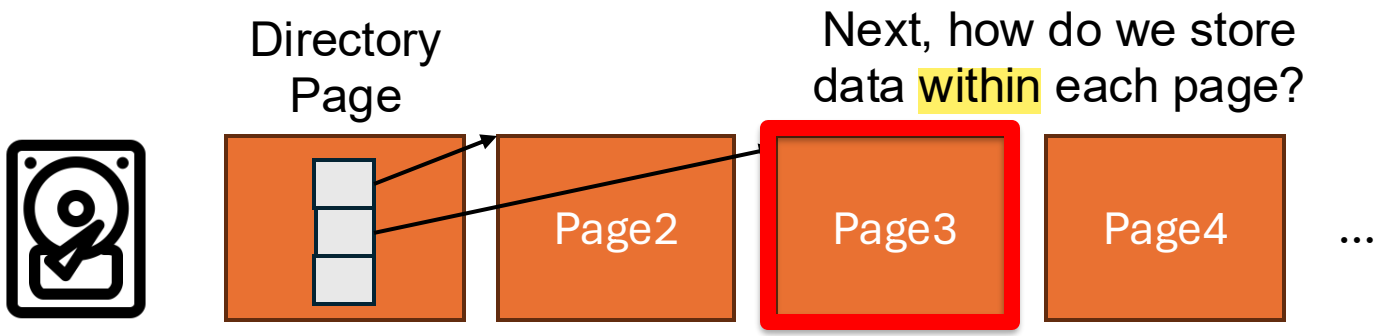
Page Storage Architecture

- Simplest idea:
  - PageID corresponds to physical location on file
  - Allocate(), Read(), Write()
  - GoDB uses this architecture
- Problem: delete
  - Cannot “delete” a physical page
  - Upper-level logic must manage Garbage Collection
  - Extra Credit Task for Lab 1!



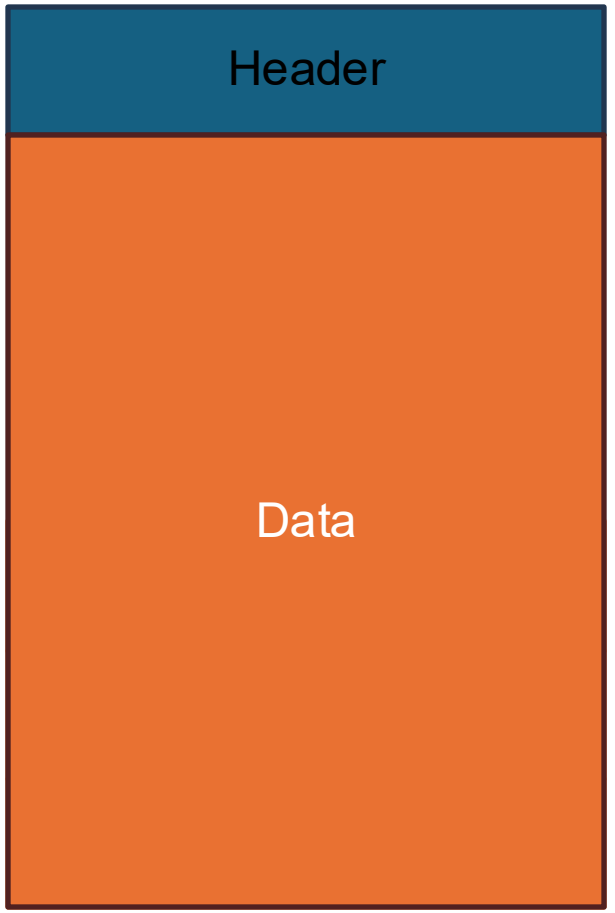
Page Storage Architecture

- Other production systems:
  - Often add a layer of indirection to map logical page IDs to physical locations
  - Often maintain extra metadata to track free space, etc.



Page designs

- Each page has some header for metadata, and the rest stores tuple data
- For now, let's consider contiguous row storage
  - A page is a bucket of tuples
  - Other types: index-oriented, log-structured

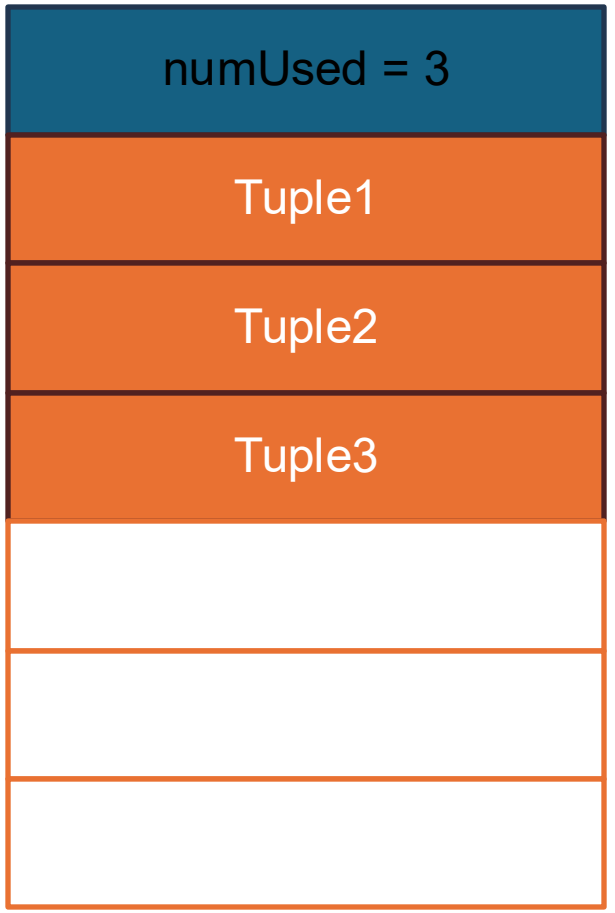


Page designs

Strawman idea: Lay tuples out contiguously?

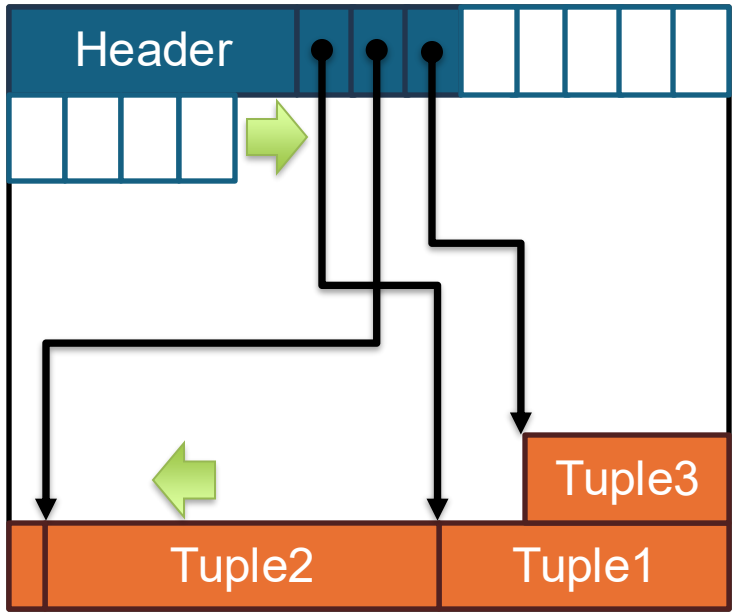
Any problems with this design?

- What about deletes?
- How to deal with variable length values?



Slotted pages

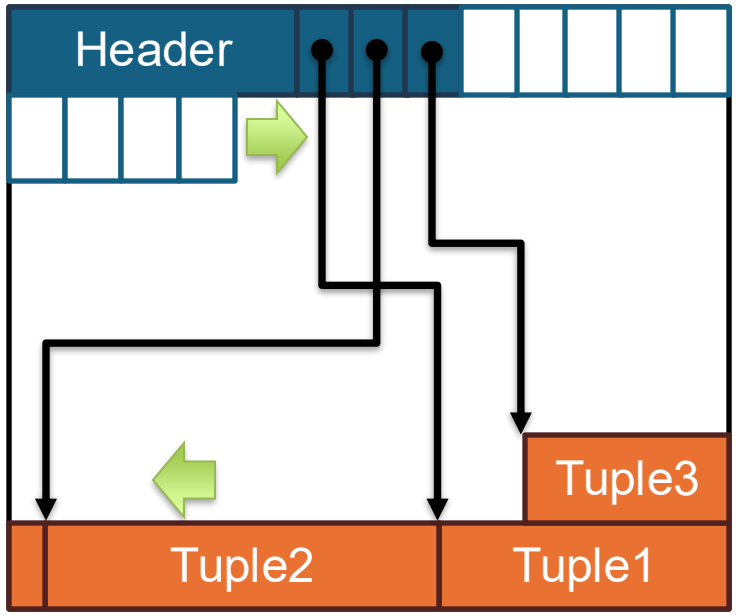
- Each page stores a “slot array” that maps slots to locations in the page
- Tuples are laid out starting from the end of the page
- Slot array and data grow towards the middle.



Slotted pages

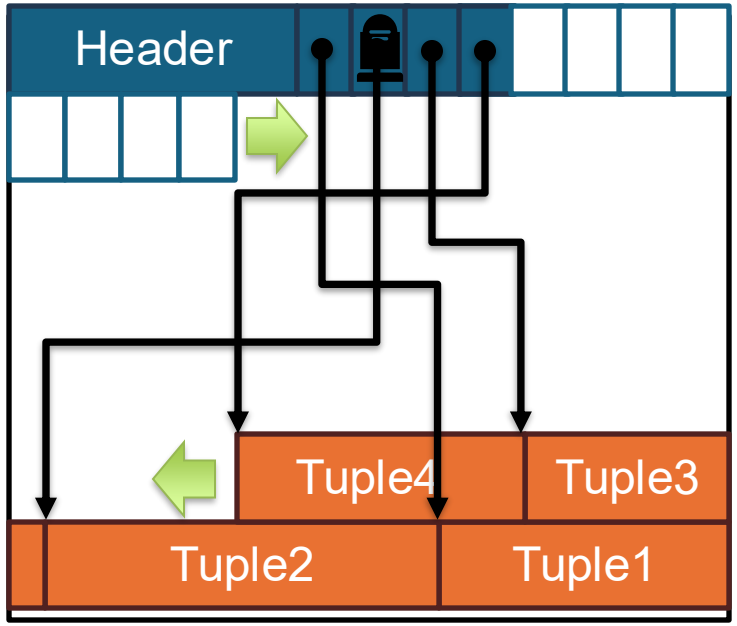
- On slotted page, a record is uniquely identified with PageID + Offset

- On read of a record ID:
  - Find location of the page
  - Request page from buffer pool
  - Find the offset in page using a slot array



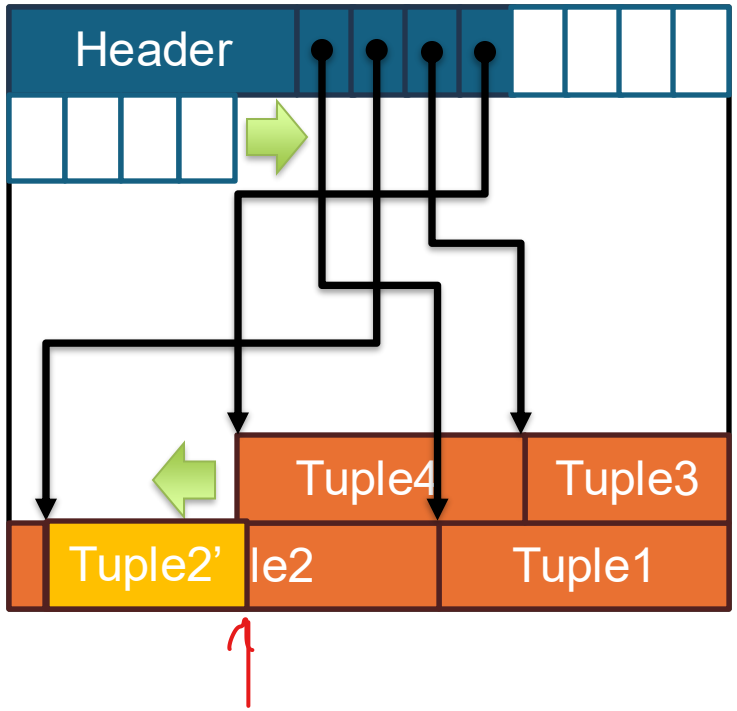
Slotted pages

- On insert:
  - Find a page with free slot
  - Check to see if tuple fit
  - If so, insert. Otherwise, retry
- On delete:
  - Invalidate tuple and remove slot array entry
  - May need a “tombstone”



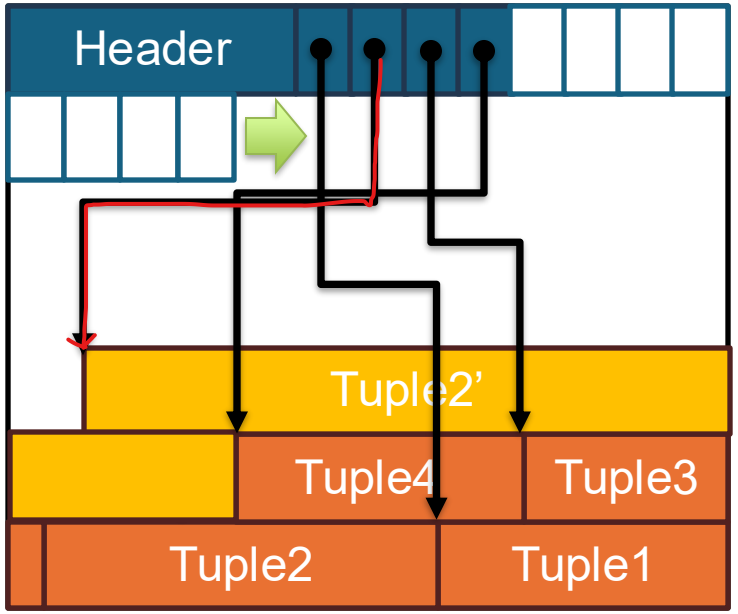
Slotted pages

- On update:
  - Find tuple
  - Overwrite data if update fits
  - Otherwise, must delete and insert into a new slot



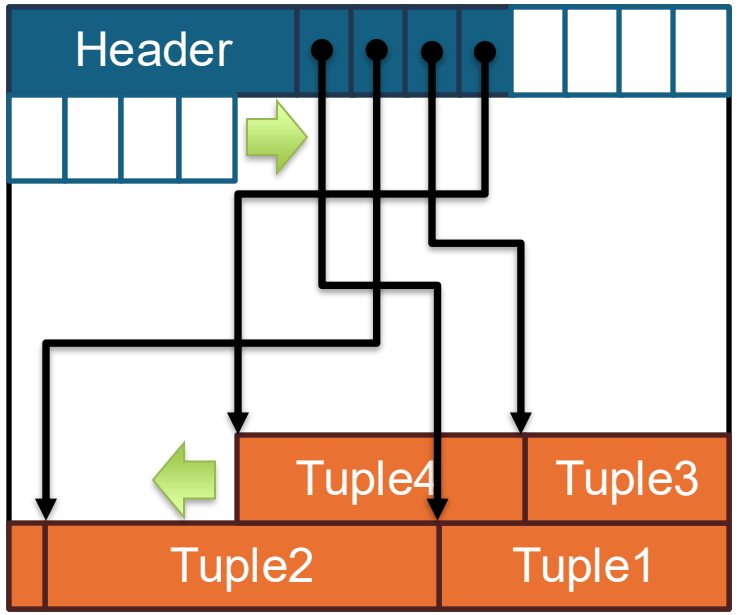
Slotted pages

- On update:
  - Find tuple
  - Overwrite data if update fits
  - Otherwise, must delete and insert into a new slot
- Note: the unique identifier of a tuple may change after update!
  - Never rely on it externally
  - May need to update other parts of the system



Slotted pages

- Problems:
  - Fragmentation
  - I/O amplification
  - Worst case: random I/O
- Production systems often need complex GC, maintenance passes, etc.
  - Newer systems often sidestep this using alternative organization like log-structured storage

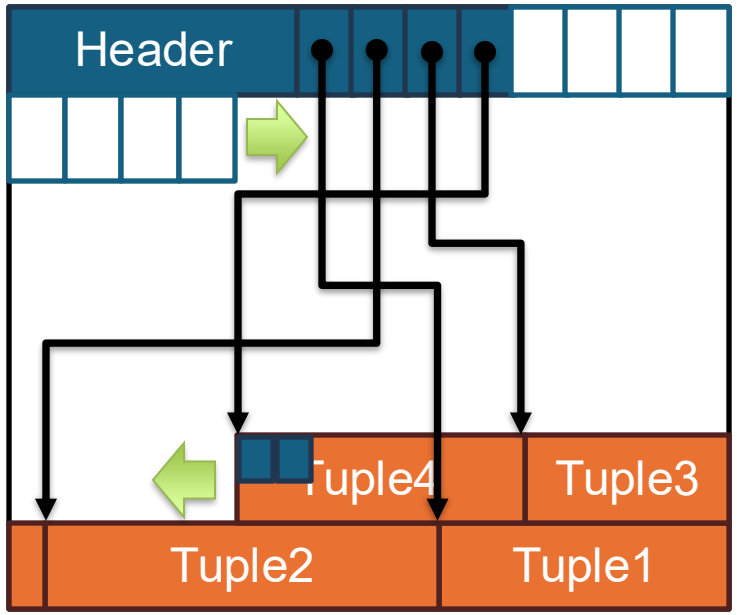


Edge Cases

- Nulls – represent "unknown" in database
  - Source of headache (more in lab 2)
  - Usually stored as a per-tuple bitmap in header
  - Sometimes represented with a special value like INT_MIN (GoDB)

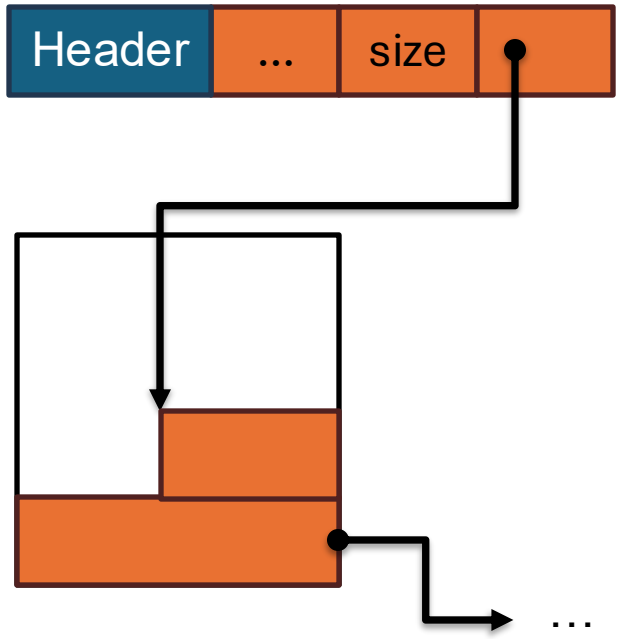
e.g.
0: not null
1: null

bitmap[0] = 1
=> column 0 is null



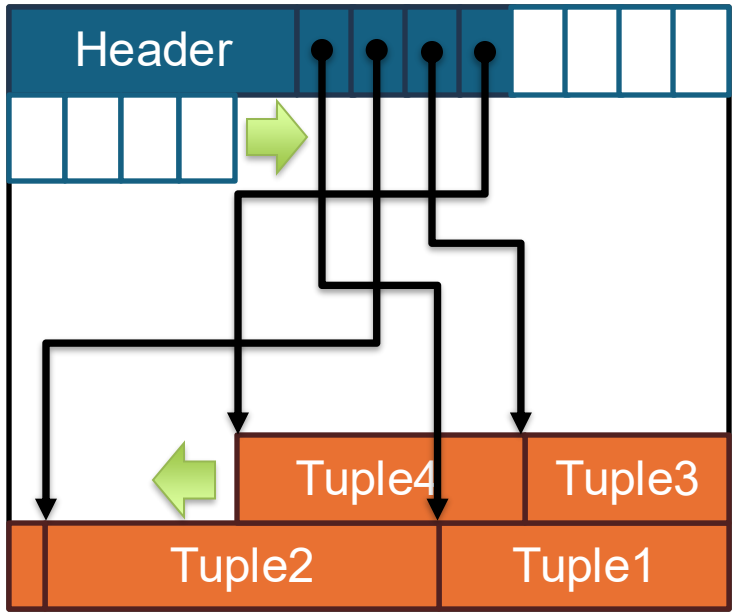
Edge Cases

- Large Values
  - Easy solution: do not allow (GoDB)
  - Otherwise – store as a linked list of overflow pages



Slotted pages

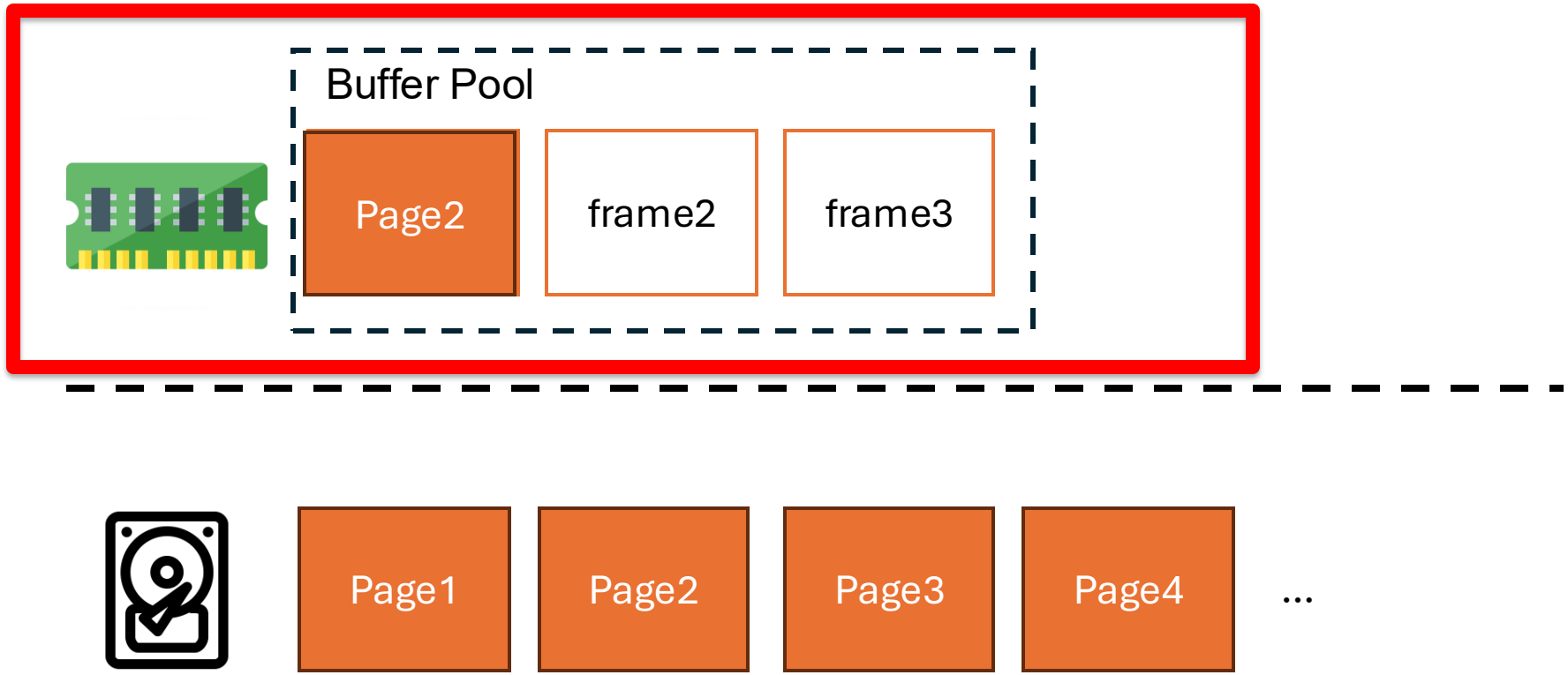
How would you simplify the layout if tuples have a fixed length (e.g., in GoDB)?



Let’s take a short break

Disk-Oriented Storage

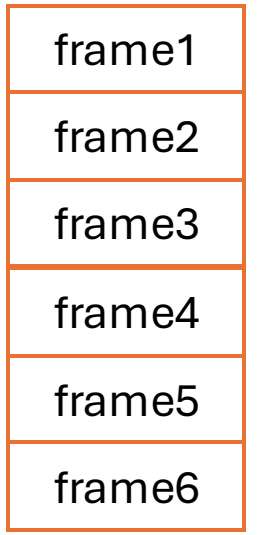
How does the DBMS manage memory and data movement?



Buffer Pool

Memory region organized as an array of fixed size pages. An array entry is called a **frame.**

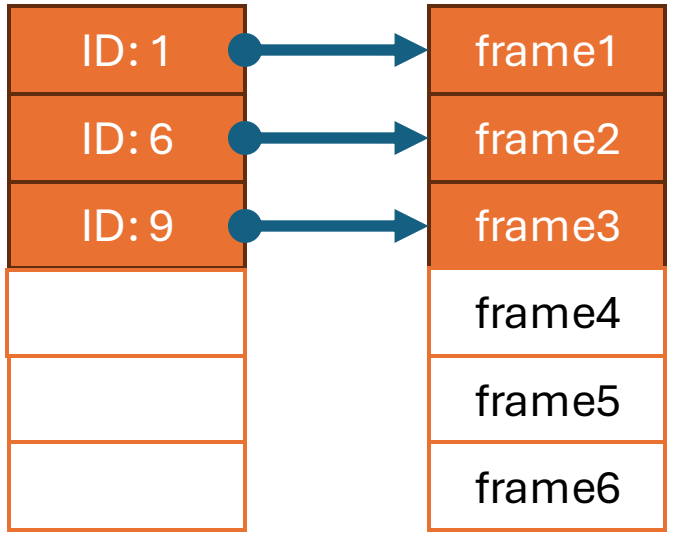
- These are usually pre-allocated and the number of frames is configurable



Buffer Pool

The **page table** keeps track of what pages are in memory

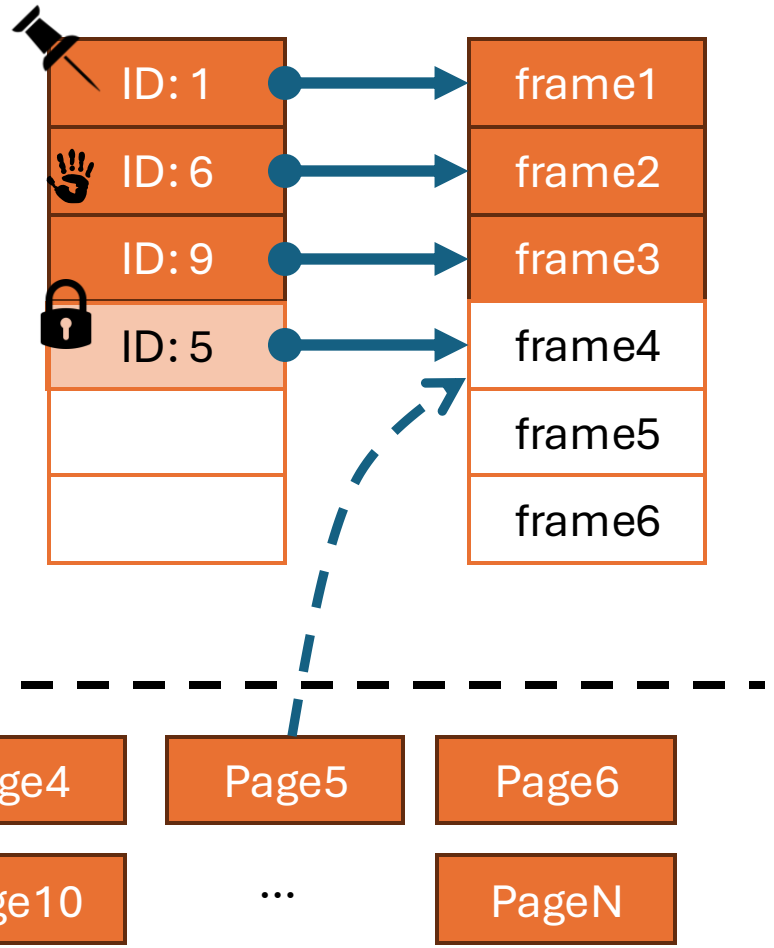
Dirty pages are kept and not written to disk immediately for performance



Buffer Pool

Extra metadata is required for correct operation:

- Dirty Flag
- Pin/Reference Counter
- Latches



Aside: Lock vs. Latches

- Lock:
  - Protect database content (e.g., rows, tables) from concurrent transactions
  - Held by user or internal transactions
- Latch:
  - Protect internal data structures (e.g., page table)
  - Held by threads
  - OS people call these “lock” or “mutex”

Hardware (e.g., SSDs) and OS (e.g., virtual memory) also use pages. They often are 4KB large.

Why do we introduce yet another paging mechanism?

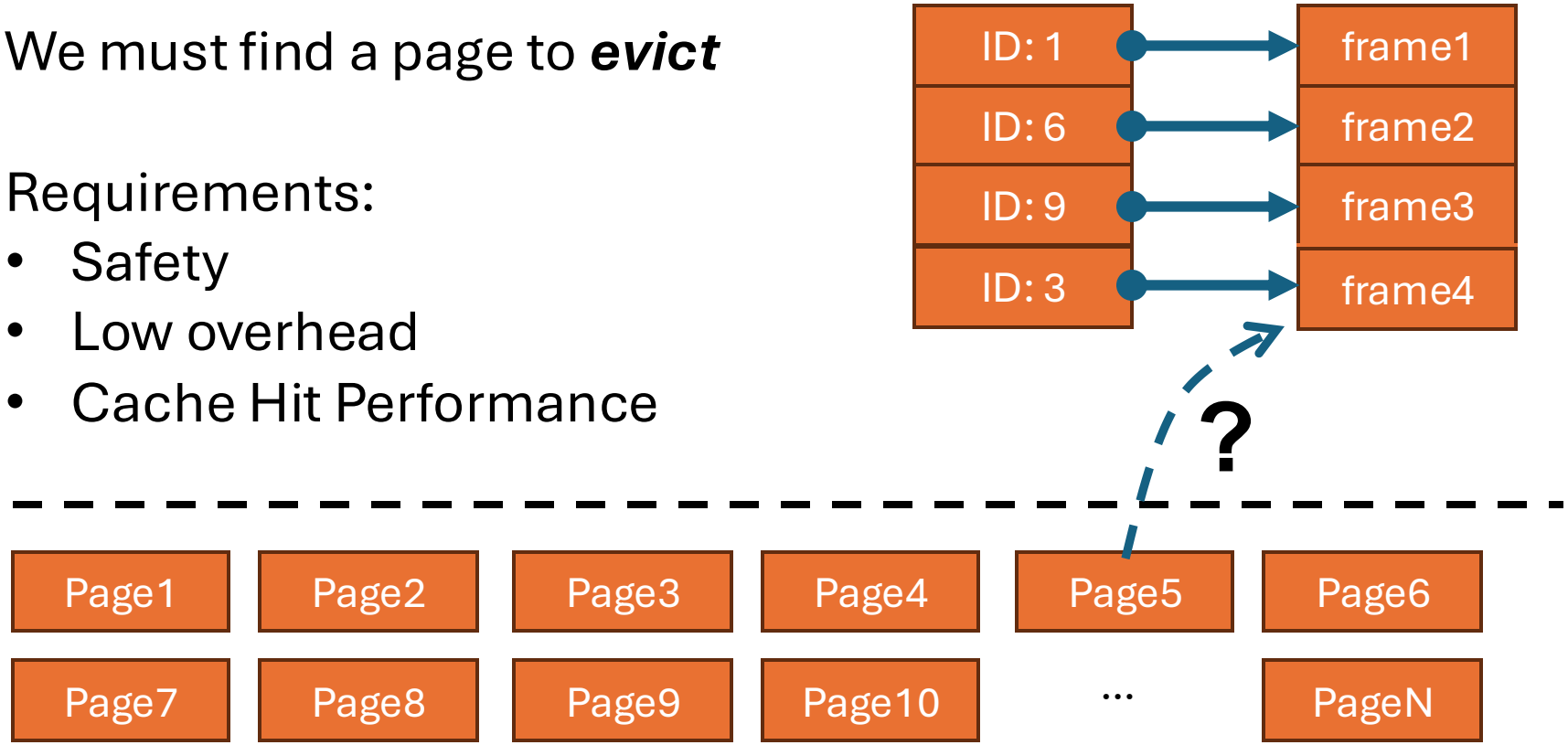
Eviction Policy

What happens when the buffer pool is full?

We must find a page to ***evict***

Requirements:

- Safety
- Low overhead
- Cache Hit Performance



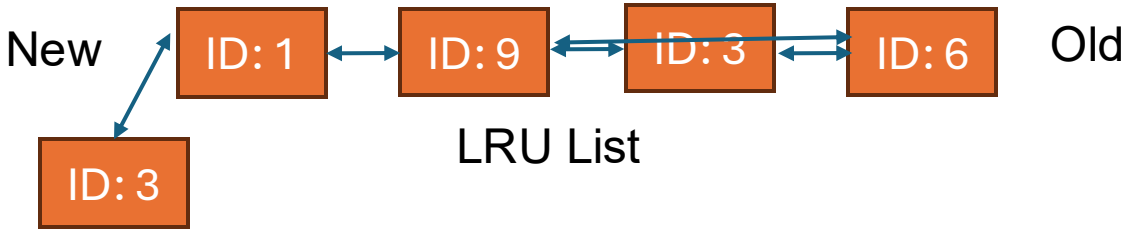
Least-Recently Used (LRU)

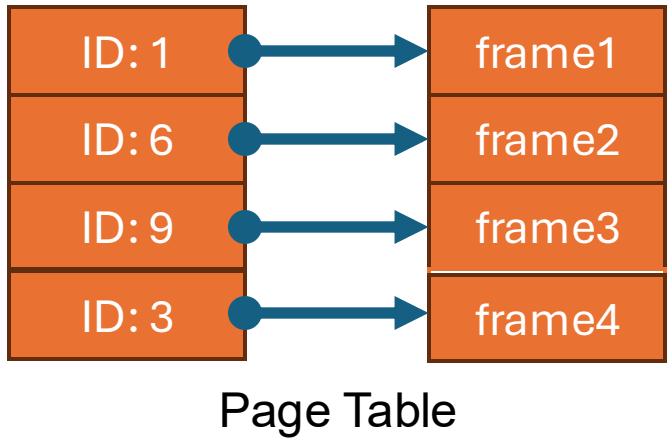
Intuitive Idea: evict the least recently used page

- Assumption: temporal locality

Implementation: Usually with a queue

- Unlink and re-insert on access

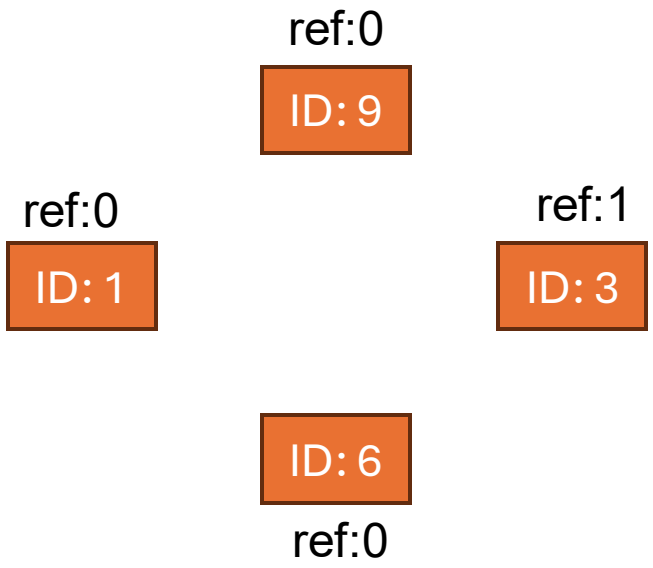


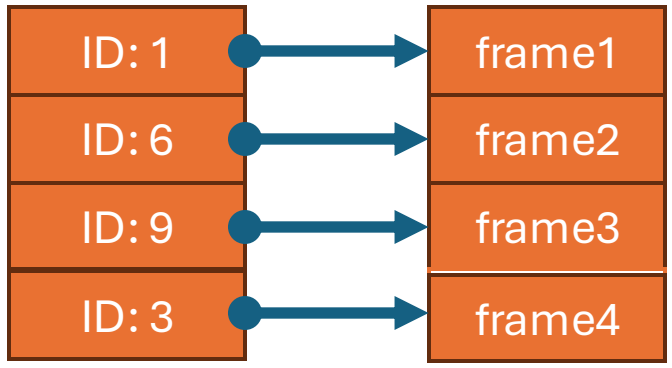


CLOCK: Approximate LRU

Idea: avoid the queue with a circular buffer & one “ref bit”

- Ideal for concurrency, hardware implementation





Page Table

CLOCK: Approximate LRU

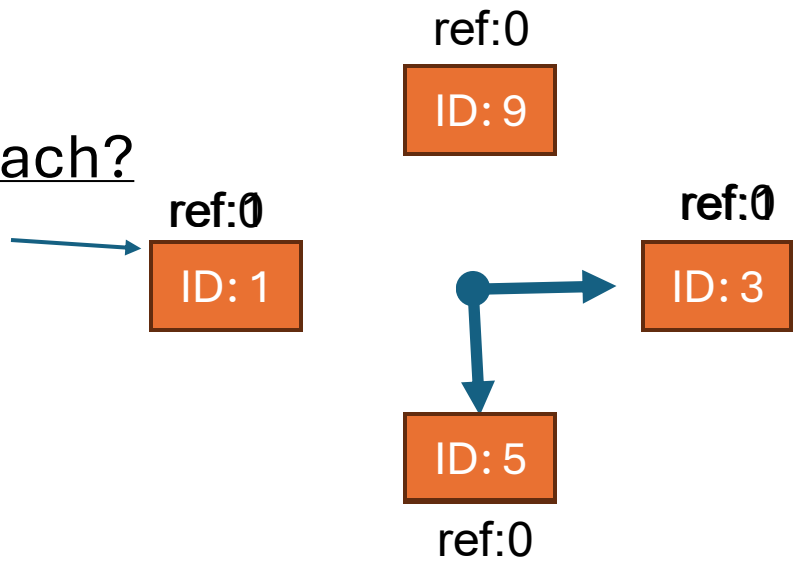
On access:

- Set ref bit

On evict:

- Unset if ref bit is set
- Evict if unset

<u>Problems with this approach?</u>



Problem: Sequential Flooding

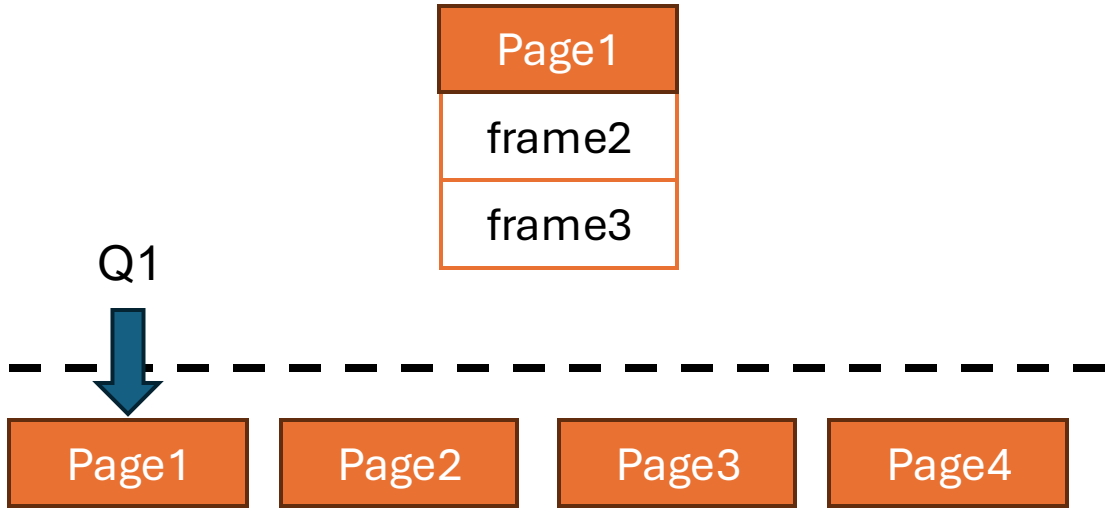
- Where does buffer pool reuse come from?
  - Skewness of data: some pages more popular than others
  - Temporal Locality
  - Spatial Locality

- In a database: a mix of point queries and sequential scans
  - Sequential scans generate many recently used pages that are never reused!
  - May cause the buffer pool to evict reusable pages

Example

Workload:

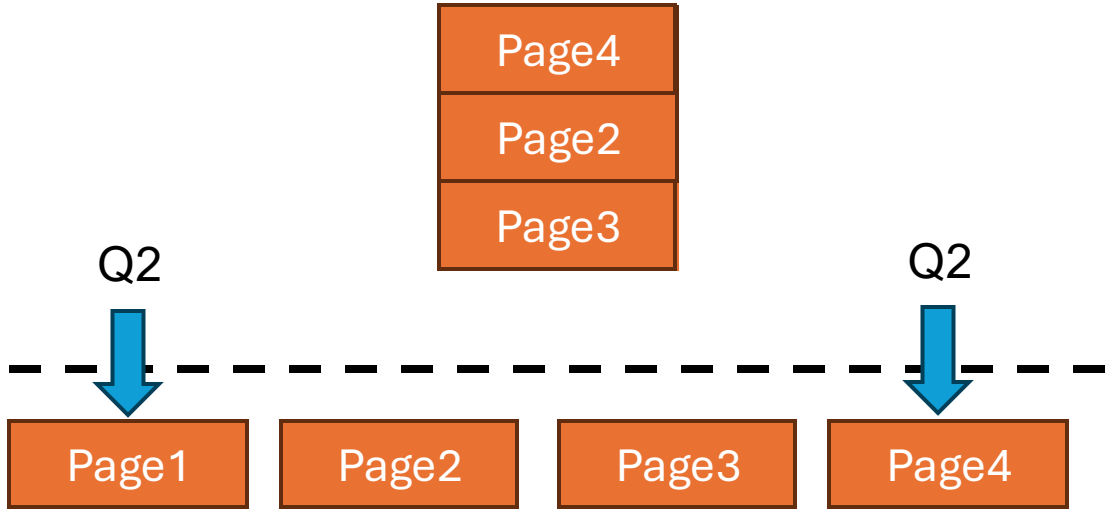
- Q1: SELECT * FROM A WHERE id = 42
- Q2: SELECT AVG(val) FROM A
- Q3: SELECT * FROM A WHERE id =42



Example

Workload:

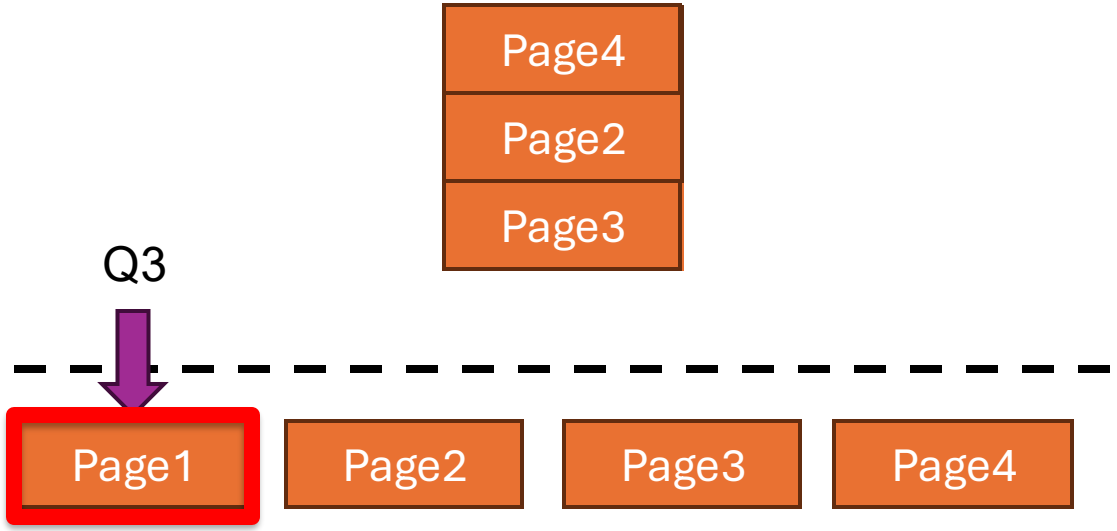
- Q1: SELECT * FROM A WHERE id = 42
- Q2: SELECT AVG(val) FROM A
- Q3: SELECT * FROM A WHERE id =42



Example

Workload:

- Q1: SELECT * FROM A WHERE id = 42
- Q2: SELECT AVG(val) FROM A
- Q3: SELECT * FROM A WHERE id =42



Least-Frequently Used

- Identify popular pages by counting access frequency
  - Avoids evicting hot pages due to scan
- Any problem you can think of?
  - Pages that are no longer hot are kept around
  - Maintenance is hard – requires O(log n) structures

LRU-K

- Track history of last K accesses
  - Replace page with oldest kth access
  - Usually, k = 2
- LRU-K implementation is usually quite complex
  - Need to track state for evicted pages
  - Linked-lists no longer suffice, need more sophisticated data structures
  - In practice, often approximated with two queues, similar to how CLOCK approximates LRU

Better Policies

What other policies can you think of?

Better Policies

- Query-local-policies: Queries often know better what the access pattern is.
- Priority Hints with programmer help
- Multiple queues (e.g., S3-FIFO)
- Approximating frequency with sketches
- And many more!
  - You will come up with your own in lab 1
  - No need to be fancy, but we will test yours against sequential flooding workloads

GoDB Lab 1

- You will build the first component of GoDB in the storage layer

- Basic APIs are provided for you
  - Autograder will only look at files in “Files to Modify”

- Due: 02/26 11:59pm
  - Write-up: 03/05 in-class
  - Start early! Lab 1 will be challenging if your systems programming and debugging skills are rusty.
  - Post on piazza / come to office hours

Next Class

- Query Processing & Access Methods
  - You will implement most of these in Lab 2

- Next Class is Feb 19!
  - Feb 17 is Monday schedule